University of Canberra
Faculty of Science and Technology

Software Technology 2

(7170)

Week 8 Tutorial & Computer Lab

Assignment 1: Individual Project

Part 5: Graph Data Structure and Algorithms in Python

Learning Objectives

- Understand graph representation using adjacency lists.
- Implement basic graph operations: add/remove vertices and edges.
- Apply BFS and DFS algorithms for traversal.
- Detect cycles in undirected graphs.
- Build a small visualization with Tkinter.
- Benchmark graph operations.

Outline of Tasks

Task	Concept	Key Points
1	Graph Structure & Modification	Adding/removing vertices & edges
2	BFS Traversal	Level-order visiting using queue
3	DFS Traversal	Depth-first visiting using stack
4	Cycle Detection	Recursion to detect cycles in undirected graphs
5	Directed & Weighted Graphs	Handling edge direction and weights
6	Visualization	GUI display
7	Benchmarking	Performance testing
8	Challenge Task	Extended Graph Visualiser

Task 1: Implement and Extend Graph Basics

Goal: Practice adding/removing vertices and edges.

- Create a Graph class in module graph.py
- Import the Graph class for and create Task 1 module (graph_tester.py)
- Add vertices A, B, C, D.
- Add edges between them, both directed and undirected (test both versions).
- Remove a vertex and an edge.
- Print the graph after each operation to observe changes.
- ***Outcome:*** Understand graph representation and basic modifications.

```
#===graph.py===
class Graph:
    def __init__(self, directed=False, weighted=False):
        self.graph = {}  # Initialize an empty dictionary to
store the adjacency list
        self.weights = {}
        self.directed = directed
        self.weighted = weighted

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []  # Add a new vertex with
an empty list of edges

    def add_edge(self, vertex1, vertex2, weight=0):
        if vertex1 in self.graph and vertex2 in self.graph: #
Check that vertices are present
            # store vertex and weight as a tuple
            self.graph[vertex1].append(vertex2)
            if self.weighted:
                self.weights[(vertex1, vertex2)] = weight
            if not self.directed: # In the case of an
undirected graph, append in both directions
                self.graph[vertex2].append(vertex1)
                if self.weighted:
                    self.weights[(vertex2, vertex1)] = weight
        else:
            print("One or both vertices not found in graph.")
```

```
#===graph_tester.py===

from graph import Graph
#Basic undirected graph tests
g = Graph()
```

```
g.add_vertex('A')
g.add_vertex('B')
g.add_vertex('C')

g.add_edge('A', 'B')
g.add_edge('B', 'C')

g.print_graph()
print()

# Directed graph tests
g = Graph(directed=True)
g.add_vertex('A')
g.add_vertex('B')
g.add_vertex('C')

g.add_edge('A', 'B')
g.add_edge('B', 'C')

g.print_graph()
print()


# Weighted graph tests
g = Graph(weighted=True, directed=True)
g.add_vertex('A')
g.add_vertex('B')
g.add_vertex('C')

g.add_edge('A', 'B', 15)
g.add_edge('B', 'C', 34)

g.print_graph()

# Removal tests
print()
print("Removal test")
removal_test = Graph()
removal_test.add_vertex('A')
removal_test.add_vertex('B')
removal_test.add_vertex('C')

removal_test.add_edge('A', 'B')
removal_test.add_edge('B', 'C')
```

```
removal_test.print_graph()

removal_test.remove_edge('A', 'B')

removal_test.print_graph()

removal_test.remove_vertex('C')

removal_test.print_graph()
print()
```

```
A --- B
B --- A C
C --- B D
D --- C

A --> B
B --> C
C --> D
D
```

```
Removal test
A --- B
B --- A C
C --- B D
D --- C
A --- B
B --- A
C --- D
D --- C
A --- B
B --- A
D
```

Task 2: Breadth First Search (BFS) Implementation

Goal: Explore graph traversal using BFS.

- Implement BFS starting from a given vertex.
- Add the following code to **graph_tester.py** to implement BFS algorithm
- Print the order of visited vertices.
- Test with a small graph (A → B → C and additional connections).
- Discuss BFS properties (level-by-level exploration).
- ***Outcome:* Grasp BFS logic and its applications.**

```
# BFS on the graph from Activity 1
print("BFS starting from vertex 'A':")
visited_bfs = g.bfs('A')
print("Visited vertices in BFS order:", visited_bfs)
```

```
BFS starting from vertex 'A':
A B C D
Visited vertices in BFS order: {'A', 'C', 'D', 'B'}
```

Breadth First Search (BFS) explores a graph level by level starting from a chosen vertex. It uses a queue, so nodes are visited in the order they are discovered. This means all neighbours are visited before moving deeper into the graph.

BFS is useful because it can find the shortest path in an unweighted graph. Its time complexity is O(V + E), making it efficient for most graph traversal tasks.

Task 3: Depth First Search (DFS) Implementation

Goal: Understand DFS differences and practice stack-based traversal.

- Implement DFS using a stack (already provided).
- Add the following code to **graph_tester.py** to implement DFS algorithm
- Compare output order with BFS on same graph.
- Test on branched graph including cycles.
- *Outcome:* Compare traversal techniques; understand search strategies.

```python
# DFS on the same graph
print("DFS starting from vertex 'A':")
visited_dfs = g.dfs('A')
print("Visited vertices in DFS order:", visited_dfs)
```

```
DFS starting from vertex 'A':
A B C D
Visited vertices in DFS order: {'A', 'C', 'D', 'B'}
```

Comparison

BFS and DFS both traverse the graph, but they do it differently. BFS goes level by level, so it visits all neighbours first, while DFS goes as deep as possible before backtracking. This means BFS usually finds the shortest path in unweighted graphs, but DFS does not guarantee that.

When testing on the same graph, the order of visited nodes is different. BFS gives a more “spread out” order, while DFS follows one path deeply before moving to others.

Task 4: Cycle Detection in Undirected Graphs

Goal: Write and test code to detect cycles using DFS recursion.

- Create graphs with and without cycles (triangle graph and linear chain).
- Add the following code to **graph_tester.py** to implement Cycle Detection algorithm in Undirectred Graphs
- Verify correct detection with boolean output.
- ***Outcome:*** **Learn how DFS can detect cycles and why this matters.**

```
#cycle detection in undirected graphs
# Create graph with a cycle
cycle_graph = Graph(directed=False)
for v in ['A', 'B', 'C']:
    cycle_graph.add_vertex(v)
cycle_graph.add_edge('A', 'B')
cycle_graph.add_edge('B', 'C')
cycle_graph.add_edge('C', 'A')  # Closing the cycle

print("Does cycle_graph have a cycle? ",
cycle_graph.has_undirected_cycle())

# Create graph without a cycle
acyclic_graph = Graph(directed=False)
for v in ['X', 'Y', 'Z']:
    acyclic_graph.add_vertex(v)
acyclic_graph.add_edge('X', 'Y')
acyclic_graph.add_edge('Y', 'Z')
print("Does acyclic_graph have a cycle? ",
acyclic_graph.has_undirected_cycle())
```

```
Does cycle_graph have a cycle?  True
Does acyclic_graph have a cycle?  False
```

Task 5: Experiment with Weighted and Directed Graphs

Goal: Practice using weighted edges and directed graphs.

- Create a weighted directed graph with given weights.
- Add the following code to **graph_tester.py** to implement Weighted and Directed Graphs
- Add edges with weights (e.g., A → B with 15, B → C with 34).
- Print graph showing weights.
- Discuss how weights affect graph operations.
- ***Outcome:*** **Understand storing and using edge weights.**

```
# Create weighted directed graph
```

```
wg = Graph(directed=True, weighted=True)
for v in ['A', 'B', 'C']:
    wg.add_vertex(v)
wg.add_edge('A', 'B', weight=15)
wg.add_edge('B', 'C', weight=34)
wg.add_edge('A', 'C', weight=50)
print("Weighted Directed Graph:")
wg.print_graph()
```

```
Weighted Directed Graph:
A --> B15 D50
B --> C34
C --> D67
D
```

Weights affect how graph operations work, especially when finding paths. Algorithms consider the total weight of edges, rather than looking for the shortest number of steps, making problems like finding the shortest path complex.

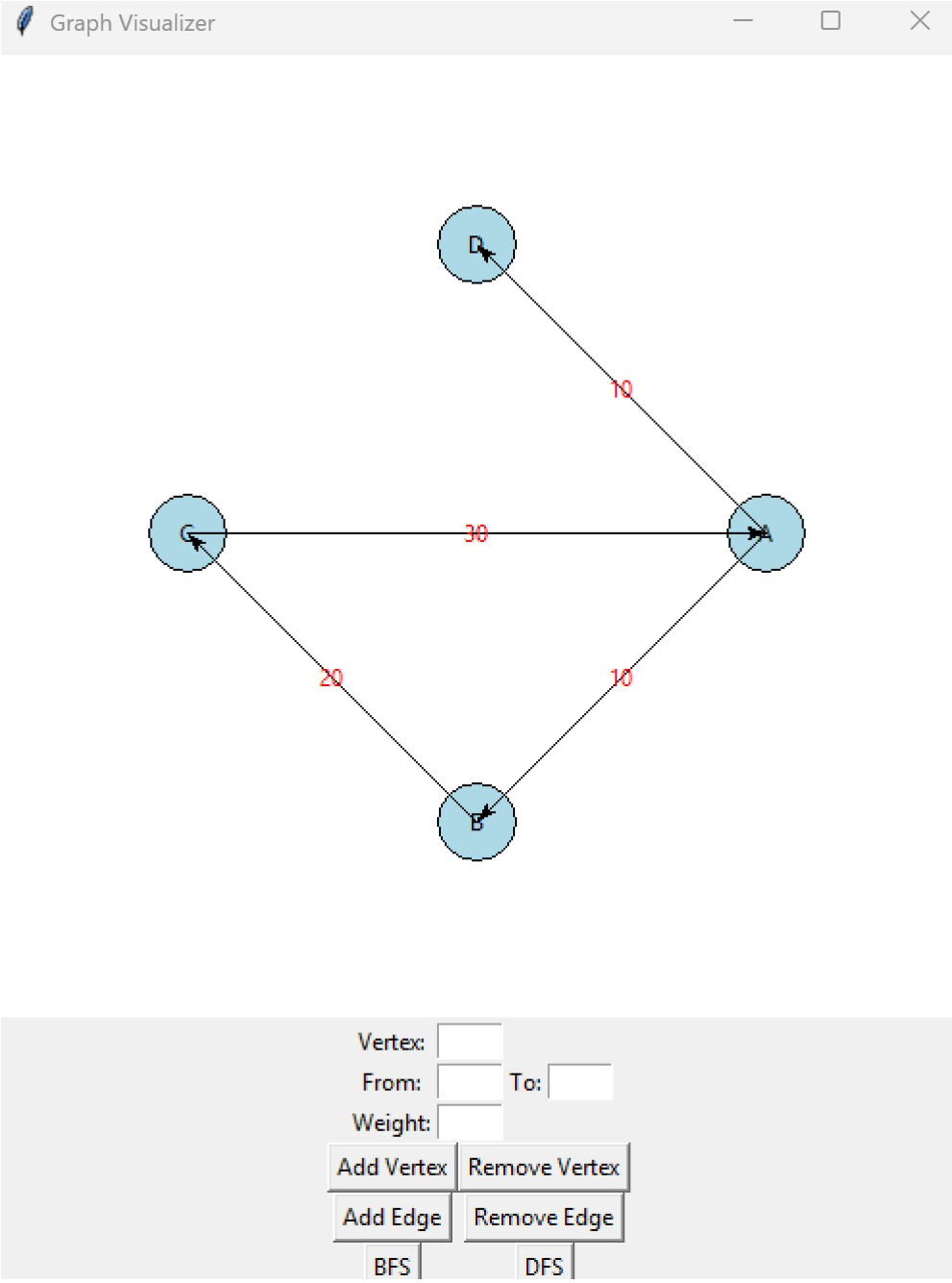
Task 6: Visualization Graph Data Structure with Tkinter

- Use Tkinter to create a simple GUI window.
- Display vertices as circles and edges as lines/arrows in a canvas.
- Implement buttons for adding/removing vertices/edges.
- Highlight BFS or DFS traversal order dynamically.
- Optional: Display edge weights near edges.
- Create a new python module (**graph_visualizer.py**) and import Graph class from graph.py

```
# === graph_visualizer.py ===

from graph import Graph
import tkinter as tk

class GraphVisualizer(tk.Tk):
  def __init__(self, graph):
    super().__init__()
    self.title("Graph Visualizer")
    self.graph = graph
    self.canvas = tk.Canvas(self, width=400, height=400, bg='white')
    self.canvas.pack()
```

```
        self.vertex_positions = {}
        self.draw_graph()

    def draw_graph(self):
        self.canvas.delete("all")
        radius = 20
        spacing = 100
        # Arrange vertices in a circle
        n = len(self.graph.graph)
        angle_gap = 360 / n if n else 0
        center_x, center_y = 200, 200
        for i, v in enumerate(self.graph.graph):
            angle = i * angle_gap
            x = center_x + spacing * tk.math.cos(tk.math.radians(angle))
            y = center_y + spacing * tk.math.sin(tk.math.radians(angle))
            self.vertex_positions[v] = (x, y)
            self.canvas.create_oval(x - radius, y - radius, x + radius, y + radius, fill='lightblue')
            self.canvas.create_text(x, y, text=v)

        # Draw edges
        for v in self.graph.graph:
            x1, y1 = self.vertex_positions[v]
            for nbr in self.graph.graph[v]:
                x2, y2 = self.vertex_positions[nbr]
                self.canvas.create_line(x1, y1, x2, y2, arrow=tk.LAST if self.graph.directed else
None)
                if self.graph.weighted:
                    w = self.graph.weights.get((v, nbr), '')
                    mid_x, mid_y = (x1 + x2) / 2, (y1 + y2) / 2
                    self.canvas.create_text(mid_x, mid_y, text=str(w), fill="red")

if __name__ == "__main__":
    g = Graph(directed=True, weighted=True)
    g.add_vertex('A')
    g.add_vertex('B')
    g.add_vertex('C')
    g.add_edge('A', 'B', 10)
    g.add_edge('B', 'C', 20)
    g.add_edge('C', 'A', 30)

    app = GraphVisualizer(g)
```

```
app.mainloop()
```



Task 7: Benchmarking Graph Operations

- Create graphs of increasing sizes (e.g., 10, 100, 1000 vertices).
- Time execution of adding vertices, adding edges, BFS, DFS.
- Print or display benchmarking results in console or GUI.
- Discuss performance implications of graph size and structure.
- Create a new python module (**graph_benchmarking.py**) and import Graph class from graph.py

```
#===graph_benchmarking.py===
```

```python
from graph import Graph
import time

def benchmark_graph_operations(num_vertices, num_edges):
    g = Graph()
    start_time = time.time()
    for i in range(num_vertices):
        g.add_vertex(str(i))
    vertex_time = time.time()
    print(f"Added {num_vertices} vertices in {vertex_time -
start_time:.4f} seconds")

    import random
    edges_added = 0
    for _ in range(num_edges):
        v1 = str(random.randint(0, num_vertices -1))
        v2 = str(random.randint(0, num_vertices -1))
        if v1 != v2:
            g.add_edge(v1, v2)
            edges_added += 1
    edge_time = time.time()
    print(f"Added {edges_added} edges in {edge_time -
vertex_time:.4f} seconds")

    # BFS from vertex '0' if exists
    if '0' in g.graph:
        bfs_start = time.time()
        g.bfs('0')
        bfs_time = time.time()
        print(f"BFS completed in {bfs_time - bfs_start:.4f}
seconds")
    else:
        print("Vertex '0' not found for BFS")

if __name__ == "__main__":
    benchmark_graph_operations(num_vertices=1000,
num_edges=3000)
```

10

```
Added 10 vertices in 0.0000 seconds
Added 2694 edges in 0.0084 seconds
0 1 9 5 2 7 3 8 6 4
BFS completed in 0.0004 seconds
```

100

```
Added 100 vertices in 0.0001 seconds
Added 2971 edges in 0.0076 seconds
0 30 32 88 37 49 10 6 51 95 20 99 70 54 92 79 25 56 22 4
 26 94 44 23 73 66 67 9 87 98 13 24 14 77 50 90 81 55 11
 76 83 93 75 61 64 12 62 38 65 35 68 17 40 21 15 57 85 9
7 42 47 86 69 8 7 27 84 78 45 72 5 31 46 71 1 48 16 91 3
3 3 80 29 74 82 43 52 89 36 58 34 96 59 63 2 18 53 28 39
 19 60 41
BFS completed in 0.0006 seconds
```

1000

```
Added 1000 vertices in 0.0002 seconds
Added 2996 edges in 0.0071 seconds
0 823 226 372 677 254 865 377 650 855 387 802 325 16 626 873 276 509 601 207 699 300 709 76 624 985 183 872 218 738 870 465 596 882 662 860 577 808 383
269 661 880 90 20 722 647 98 238 584 93 947 200 487 412 866 424 181 110 215 251 954 285 780 651 208 592 293 155 983 115 255 859 563 977 726 735 477 393
653 460 446 498 720 526 82 495 960 888 384 891 623 303 158 807 629 578 198 113 756 915 557 912 655 409 231 284 455 199 450 438 828 679 478 611 453 74 60
4 366 410 130 307 749 966 572 680 752 637 314 120 248 473 357 892 118 111 553 260 456 583 420 514 499 11 547 225 723 881 727 347 185 933 445 952 309 585
 877 451 214 953 764 355 301 787 138 190 542 937 25 824 964 686 945 406 201 50 273 223 988 627 352 302 940 153 515 103 99 429 22 558 607 546 265 506 833
 386 929 967 249 163 766 188 196 290 909 175 404 19 348 6 830 718 518 914 525 63 687 491 633 167 731 173 141 800 197 522 794 242 297 837 435 878 84 100
693 974 815 362 970 772 341 464 976 49 346 403 799 579 812 889 327 77 121 434 586 44 950 857 782 869 313 574 609 135 315 750 639 137 620 600 841 31 408
339 240 217 702 769 908 247 541 236 751 41 331 696 501 304 820 275 177 602 666 230 169 232 744 605 784 282 18 520 528 108 338 389 195 747 87 926 836 846
 648 555 597 505 742 233 535 911 136 102 467 760 910 763 981 632 407 85 978 625 5 827 548 152 705 733 211 274 75 56 552 241 330 901 923 59 818 305 995 5
10 39 367 621 184 292 38 668 863 992 567 116 373 147 893 212 918 345 86 496 140 593 97 104 402 385 821 422 262 817 757 266 430 310 935 159 128 40 550 65
9 83 257 295 721 906 264 773 48 615 224 428 982 921 194 765 968 811 938 444 287 919 813 468 278 894 532 411 949 379 630 774 905 354 955 145 344 755 540
5 442 683 814 714 418 688 394 294 288 674 603 849 476 564 684 695 993 221 471 789 480 318 642 728 391 28 984 106 154 150 972 847 454 21 975 536 203 898
364 351 160 337 37 123 560 986 482 329 871 209 862 109 234 280 776 336 523 826 95 475 171 783 638 554 174 569 333 941 712 261 439 963 51 996 965 899 484
 581 127 170 785 927 851 533 392 79 570 58 360 671 672 258 227 179 252 417 462 178 164 17 432 854 549 502 441 646 316 399 122 180 582 835 72 636 786 876
 716 308 469 119 472 149 619 368 296 987 186 213 1 96 437 793 204 864 614 753 850 182 848 165 838 397 53 832 61 370 924 427 416 461 8 470 997 922 980 12
5 942 973 936 781 801 143 401 319 512 875 989 490 381 7 932 320 70 775 321 700 697 797 91 272 10 656 517 151 640 883 768 990 673 907 34 3 146 253 810 44
0 717 770 513 805 861 239 349 474 101
BFS completed in 0.0014 seconds
```

By increasing graph size the BFS increases in time, therefore for simulations or code that utilises higher magnitudes of vertices the time taken to complete operations will increase.

By understanding structure, if there are fewer edges it would result in faster traversals and less memory usage and vice versa if the graph contained many edges.

Task 8: Challenge Task- Extended Graph Data Structure Visualizer

- **Implement and Visualize Directed Cycle Detection (Cycle detection in directed graphs)**
- The provided graph visualizer in Task 6 includes cycle detection only for undirected graphs. Detecting cycles in directed graphs is a bit different — it requires tracking recursion stack to detect back edges.
- Task Requirements:
    - Extend the GraphVisualizer (task 6) class with to **detect and visualize cycles in both directed and undirected graphs**.
- Test each of the button shown in the starter template code for the graph_visualiser app shown below.

```
#===starter_template_code ===
```

```
#===extended_graph_visualiser.py

#===this is incomplete code which you need to complete for this
#challenge task

from graph import Graph
import tkinter as tk
import math
import time
import threading

class GraphVisualizer(tk.Tk):
    def __init__(self, graph):
        super().__init__()
        self.title("Graph Visualizer")
        self.geometry('650x650')
        self.graph = graph
        self.canvas = tk.Canvas(self, width=600, height=600,
bg='white')
        self.canvas.pack(pady=10)

        self.vertex_positions = {}
        self.node_radius = 20
        self.spacing = 180
        self.center = (300, 300)
        self.vertex_circles = {}  # vertex -> circle id
        self.edge_lines = {}      # (v, nbr) -> line id

        control_frame = tk.Frame(self)
        control_frame.pack()

        btn_bfs = tk.Button(control_frame, text="Run BFS",
command=lambda: self.run_traversal("bfs"))
        btn_bfs.pack(side=tk.LEFT, padx=5)

        btn_dfs = tk.Button(control_frame, text="Run DFS",
command=lambda: self.run_traversal("dfs"))
        btn_dfs.pack(side=tk.LEFT, padx=5)

        btn_cycle_undirected = tk.Button(control_frame, text="Detect
Undirected Cycle",

command=self.run_cycle_detection)
        btn_cycle_undirected.pack(side=tk.LEFT, padx=5)

        btn_cycle_directed = tk.Button(control_frame, text="Detect
Directed Cycle",

command=self.run_directed_cycle_detection)
```

```
        btn_cycle_directed.pack(side=tk.LEFT, padx=5)

        btn_reset = tk.Button(control_frame, text="Reset Colors",
command=self.reset_colors)
        btn_reset.pack(side=tk.LEFT, padx=5)

        self.info_label = tk.Label(self, text="")
        self.info_label.pack(pady=5)

        self.draw_graph()

    def draw_graph(self):
        self.canvas.delete("all")
        self.vertex_positions.clear()
        self.vertex_circles.clear()
        self.edge_lines.clear()

        vertices = list(self.graph.graph.keys())
        n = len(vertices)
        if n == 0:
            return

        angle_gap = 360 / n
        cx, cy = self.center

        # Position vertices in a circle
        for i, v in enumerate(vertices):
            angle = i * angle_gap
            x = cx + self.spacing * math.cos(math.radians(angle))
            y = cy + self.spacing * math.sin(math.radians(angle))
            self.vertex_positions[v] = (x, y)
            circle_id = self.canvas.create_oval(x -
self.node_radius, y - self.node_radius,
                                                x +
self.node_radius, y + self.node_radius,
                                                fill='lightblue',
outline='black', width=2)
            self.vertex_circles[v] = circle_id
            self.canvas.create_text(x, y, text=v, font=("Arial", 12,
"bold"))

        # Draw edges
        for v in vertices:
            x1, y1 = self.vertex_positions[v]
            for nbr in self.graph.graph[v]:
                x2, y2 = self.vertex_positions[nbr]

                dx, dy = x2 - x1, y2 - y1
```

```
                dist = math.sqrt(dx*dx + dy*dy)
                if dist == 0:
                    continue
                offset_x = dx / dist * self.node_radius
                offset_y = dy / dist * self.node_radius
                start = (x1 + offset_x, y1 + offset_y)
                end = (x2 - offset_x, y2 - offset_y)

                if self.graph.directed:
                    line_id = self.canvas.create_line(start[0],
start[1], end[0], end[1], arrow=tk.LAST, width=2)
                else:
                    line_id = self.canvas.create_line(start[0],
start[1], end[0], end[1], width=2)

                self.edge_lines[(v, nbr)] = line_id

                if self.graph.weighted:
                    w = self.graph.weights.get((v, nbr), '')
                    mid_x, mid_y = (start[0] + end[0]) / 2,
(start[1] + end[1]) / 2
                    self.canvas.create_text(mid_x, mid_y,
text=str(w), fill="red", font=("Arial", 10, "italic"))

    def reset_colors(self):
        for v, circle_id in self.vertex_circles.items():
            self.canvas.itemconfig(circle_id, fill='lightblue')
        for edge_id in self.edge_lines.values():
            self.canvas.itemconfig(edge_id, fill='black', width=2)
        self.info_label.config(text="")

    def highlight_vertex(self, vertex, color):
        circle_id = self.vertex_circles.get(vertex)
        if circle_id:
            self.canvas.itemconfig(circle_id, fill=color)
            self.update()

    def highlight_edge(self, v1, v2, color):
        line_id = self.edge_lines.get((v1, v2))
        if line_id:
            self.canvas.itemconfig(line_id, fill=color, width=3)
            self.update()

    def run_traversal(self, method):
        def worker():
            self.reset_colors()
            visited = set()
```

```
            start_vertex = 'A' if 'A' in self.graph.graph else
next(iter(self.graph.graph), None)
            if not start_vertex:
                self.info_label.config(text="Graph is empty")
                return

            if method == "bfs":
                queue = [start_vertex]
                self.info_label.config(text="Running BFS...")
                while queue:
                    vertex = queue.pop(0)
                    if vertex not in visited:
                        self.highlight_vertex(vertex, 'orange')
                        self.info_label.config(text=f"BFS visiting:
{vertex}")
                        visited.add(vertex)
                        time.sleep(0.7)

                        for nbr in self.graph.graph[vertex]:
                            if nbr not in visited and nbr not in
queue:
                                queue.append(nbr)
                                self.highlight_edge(vertex, nbr,
'green')
                                time.sleep(0.3)
                self.info_label.config(text="BFS complete.")

            elif method == "dfs":
                stack = [start_vertex]
                self.info_label.config(text="Running DFS...")
                while stack:
                    vertex = stack.pop()
                    if vertex not in visited:
                        self.highlight_vertex(vertex, 'purple')
                        self.info_label.config(text=f"DFS visiting:
{vertex}")
                        visited.add(vertex)
                        time.sleep(0.7)

                        for nbr in
reversed(self.graph.graph[vertex]):
                            if nbr not in visited:
                                stack.append(nbr)
                                self.highlight_edge(vertex, nbr,
'blue')
                                time.sleep(0.3)
                self.info_label.config(text="DFS complete.")
```

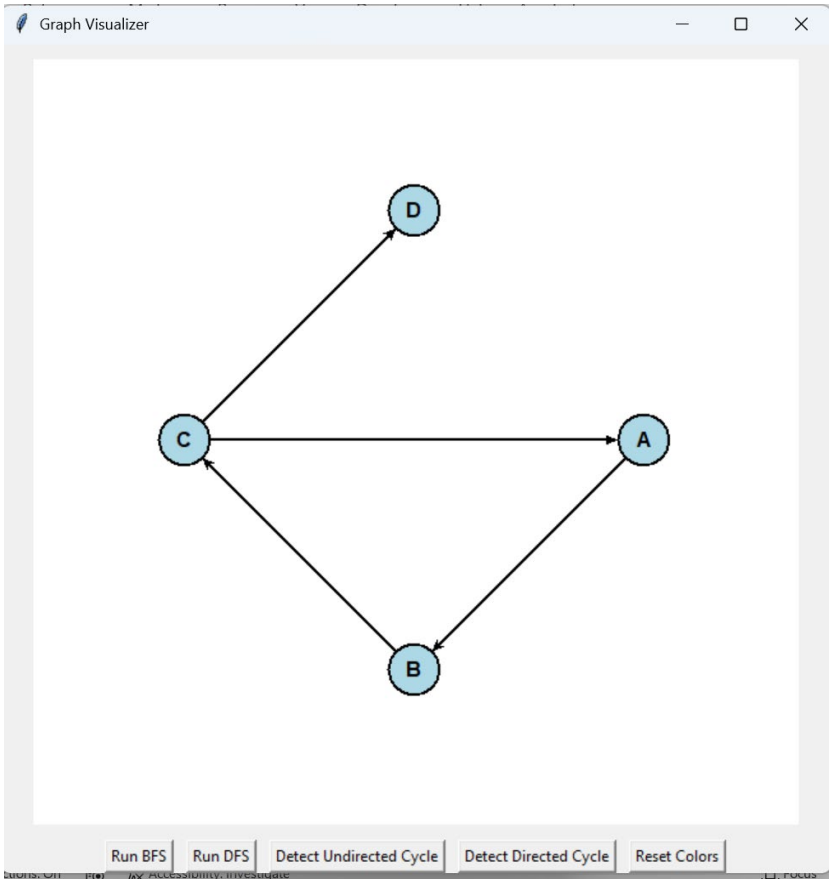
```
        threading.Thread(target=worker, daemon=True).start()

    def run_cycle_detection(self):
        self.reset_colors()
        self.info_label.config(text="Detecting cycles in undirected
graph...")

        if self.graph.directed:
            self.info_label.config(text="Cycle detection for
directed graph is not implemented.")
            return

        visited = set()
        has_cycle = False

        def cycle_dfs(current, visited_set, parent):
            visited_set.add(current)
            self.highlight_vertex(current, 'yellow')
            self.update()
            time.sleep(0.5)

            for nbr in self.graph.graph[current]:
                if nbr not in visited_set:
                    self.highlight_edge(current, nbr, 'orange')
                    self.update()
                    time.sleep(0.4)
                    if cycle_dfs(nbr, visited_set, current):
                        return True
                elif parent != nbr:
                    self.highlight_edge(current, nbr, 'red')
                    self.highlight_vertex(nbr, 'red')
                    self.highlight_vertex(current, 'red')
                    self.update()
                    time.sleep(0.5)
                    return True
            return False

        for vertex in self.graph.graph:
            if vertex not in visited:
                if cycle_dfs(vertex, visited, None):
                    has_cycle = True
                    break

        if has_cycle:
            self.info_label.config(text="Cycle detected in the
graph!")
        else:
```

```
            self.info_label.config(text="No cycles found in the
graph.")

    def run_directed_cycle_detection(self):
        self.reset_colors()
        if not self.graph.directed:
            self.info_label.config(text="Directed cycle detection
works only on directed graphs.")
            return

        self.info_label.config(text="Detecting cycles in directed
graph...")
        visited = set()
        rec_stack = set()
        has_cycle = False

        def dfs(vertex):
            nonlocal has_cycle
            visited.add(vertex)
            rec_stack.add(vertex)
            self.highlight_vertex(vertex, 'yellow')
            self.update()
            time.sleep(0.5)

            for nbr in self.graph.graph[vertex]:
                if nbr not in visited:
                    self.highlight_edge(vertex, nbr, 'orange')
                    self.update()
                    time.sleep(0.5)
                    if dfs(nbr):
                        self.highlight_edge(vertex, nbr, 'red')
                        self.highlight_vertex(nbr, 'red')
                        self.highlight_vertex(vertex, 'red')
                        self.update()
                        time.sleep(0.5)
                        return True
                elif nbr in rec_stack:
                    self.highlight_edge(vertex, nbr, 'red')
                    self.highlight_vertex(nbr, 'red')
                    self.highlight_vertex(vertex, 'red')
                    self.update()
                    time.sleep(0.5)
                    return True

            rec_stack.remove(vertex)
            return False

        for vertex in self.graph.graph:
```

```
            if vertex not in visited:
                if dfs(vertex):
                    has_cycle = True
                    break

        if has_cycle:
            self.info_label.config(text="Cycle detected in the
directed graph!")
        else:
            self.info_label.config(text="No cycles found in the
directed graph.")

if __name__ == "__main__":
    from collections import defaultdict

    # Minimal Graph class implementation for testing
    class Graph:
        def __init__(self, directed=False, weighted=False):
            self.graph = defaultdict(list)
            self.weights = {}
            self.directed = directed
            self.weighted = weighted

        def add_vertex(self, v):
            if v not in self.graph:
                self.graph[v] = []

        def add_edge(self, v1, v2, weight=0):
            if v1 in self.graph and v2 in self.graph:
                self.graph[v1].append(v2)
                if self.weighted:
                    self.weights[(v1, v2)] = weight
                if not self.directed:
                    self.graph[v2].append(v1)
                    if self.weighted:
                        self.weights[(v2, v1)] = weight

    # Create a directed graph with a cycle: A->B->C->A and extra
edge C->D
    g = Graph(directed=True, weighted=False)
    for v in ['A', 'B', 'C', 'D']:
        g.add_vertex(v)
    g.add_edge('A', 'B')
    g.add_edge('B', 'C')
    g.add_edge('C', 'A')  # Cycle here
    g.add_edge('C', 'D')

    app = GraphVisualizer(g)
```

```
app.mainloop()
```

A successful test run of this starter template code should produces the following output:



- Make a note of the results for `Run BFS, Run BFS, Detect Undirected Cycle, Detect Directed Cycle` and `Reset Colors` button presses.
- You will notice that one of the button (Detected Undirected Cycle) button does not work.
- The challenge task for you is to fix this problem so that you can get both the **`detect undirected cycle`** and **`detect directed cycle`** button working in this app.
- The current starter template code allows :
    - Implements DFS recursion and cycle detection logic for directed graphs.
    - Allows you to practice visualization and UI updates.
    - Encourages thinking about recursion stacks for graph algorithms.
    - Builds on code you have already seen.
    - The starter code already implements directed cycle detection code and includes:

        - A button to trigger directed cycle detection.
        - Animated DFS recursion with recursion stack tracking.
        - Highlights vertices (yellow) and edges visited (orange) and those in the cycle (red).
        - Also provided Informative status messages in the UI, when cycle is detected.
- What is missing in the starter code and what exactly needs to be implemented by you to complete this visualiser code for challenge task?
    - Update the code to make **both directed and undirected cycle detection** buttons working for the GraphVisualizer App
    - you need to be able to run the visualizer with two different graphs:
        - **Undirected graph** to test the undirected cycle detection button.
        - **Directed graph** to test the directed cycle detection button.
        - Because the visualizer depends on the structure of the graph passed into it, you can:
            - **Create and run the app with an undirected graph** when you want to test undirected cycle detection.
            - **Create and run the app with a directed graph** when you want to test directed cycle detection.
        - **Report the results of the completed task with updated code and testing screenshots below:**